

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..14

BACKWARD COMPATIBILITY

 Copy as Markdown

## Backward compatibility

As your app evolves, features will be added or changed. Your users may still run older versions of the app, since not everyone updates and app store updates take time. Therefore it may be essential to make updates to your server compatible with older app versions.

Following a simple set of rules, your server will stay compatible with older app versions:

1. **Avoid changing parameter names in endpoint methods.** In the API Serverpod generates, the parameters are passed by name. This means that changing the parameter names of the endpoint methods will break backward compatibility.
2. **Do not delete endpoint methods or change their signature.** Instead, add new methods if you must pass another set of parameters. Technically, you can add new named parameters if they are not required, but creating a new method may still feel cleaner.
3. **Avoid changing or removing fields and types in the serialized classes.** However, you are free to add new fields as long as they are nullable or have a default value.
4. **Do not rename registered future calls.** Queued invocations are stored in the database under the future call's registered name, so a rename breaks calls that were scheduled before the update.
5. **Keep future call payload models compatible.** A future call receives a single serializable object, and queued invocations were serialized with the old model version, so the model rules above apply to it too.

For polymorphic models, Serverpod can also deserialize an unknown subtype as its known base class, which lets you add subtypes without breaking older clients. See [Handling unknown class names](#).

## Managing breaking changes with endpoint inheritance

An endpoint sub-class can be useful when you have to make a breaking change to an entire endpoint to keep supporting existing clients. Doing so allows you to share most of its implementation with the endpoint. For the mechanics of endpoint inheritance and the `@doNotGenerate` annotation, see [Endpoint inheritance](#).



Ask AI

Imagine you had a "team" management endpoint where before a user could join if they had an e-mail address ending in the expected domain, but now it should be opened up for anyone to join if they can provide an "invite code". Additionally, the return type (serialized classes) should be updated across the entire endpoint, which would not be allowed on the existing one.

Transitioning from the current to the new endpoint structure might look like this:

```
@Deprecated('Use TeamV2Endpoint instead')
class TeamEndpoint extends Endpoint {
  Future<TeamInfo> join(Session session) async {
    // ...
  }

  // many more methods, like `leave`, etc.
}

class TeamV2Endpoint extends TeamEndpoint {
  @override
  @doNotGenerate
  Future<TeamInfo> join(Session session) async {
    throw UnimplementedError();
  }

  Future<NewTeamInfo> joinWithCode(Session session, String invitationCode) async {
    // ...
  }
}
```

In the above example, we created a new `TeamV2` endpoint, which hides the `join` method and instead exposes a `joinWithCode` method with the added parameter and the new return type. Additionally, all the other inherited (and untouched) methods from the parent class are exposed.

While we may have liked to re-use the `join` method name, Dart inheritance rules do not allow doing so. Otherwise, we would have to write the endpoint from scratch, meaning without inheritance, and re-implement all methods we would like to keep.

In your client, you could then move all usages from `client.team` to `client.teamV2` and eventually (after all clients have upgraded) remove the old endpoint on the server. That means either marking the old endpoint with `@doNotGenerate` on the class or deleting it and moving the re-used method implementations you want to keep to the new V2 endpoint class.

An alternative pattern to consider would be to move all the business logic for an endpoint into a helper class and then call into that from the endpoint. A V2 endpoint can then reuse the business logic through the helper class without subclassing the old endpoint. This has the added benefit of the endpoint class clearly listing all exposed methods, and you don't have to wonder what you inherit from the base class.